Sequence Modeling – Section 1: Recurrent Neural Networks (RNN)

Introduction

Neural networks that we use for images or tabular data process each input **independently** – they have no memory of what came before. But the real world is full of **sequential data**: a temperature reading depends on yesterday's temperature, a word's meaning depends on the words before it, a stock price depends on its history.

This section builds a complete understanding of **Recurrent Neural Networks (RNN)** – the architecture that gives neural networks memory. We will:

- Understand how RNNs process sequences step by step
- Walk through the **exact math** at every time step with real numbers
- Visualize the information flow inside the network
- Build and interpret an RNN using Keras on a real time series dataset

---

Part 1: What Is an RNN?

What It Is

A standard (feedforward) neural network takes an input, processes it, and produces an output – then **forgets everything**. It has no notion of "what happened before."

A **Recurrent Neural Network** solves this by passing a **hidden state** – a memory vector – from one time step to the next. At each step, the RNN reads the current input **and** the memory from the previous step, updates the memory, and optionally produces an output.

Real-world analogy: Imagine reading a sentence word by word while keeping a mental note of the context. When you read "The animal didn't cross the street because **it** was too tired," you know "it" refers to "the animal" – because you remember the earlier words. Your brain is doing exactly what an RNN does: processing each word while maintaining a running context (hidden state) that summarizes everything seen so far.

Why It Matters

Data Type	Example	Why sequence matters
Time series	Temperature, stock prices	Current value depends on past values
Natural language	Sentences, documents	Word meaning depends on context
Audio	Speech signals	Sounds depend on preceding sounds
Video	Frame sequences	Current frame depends on motion history

The Core Recurrence Formula

Intuition first: At every time step t, the RNN combines two things – the **current input** x_t and its **current memory** h_{t-1} – to compute a **new memory** h_t. That new memory is then passed to the next step.

Formula:

$$h_t = \tanh(W_h \cdot h_{t-1} + W_x \cdot x_t + b_h)$$

$$\hat{y}_t = W_y \cdot h_t + b_y$$

Every symbol defined:

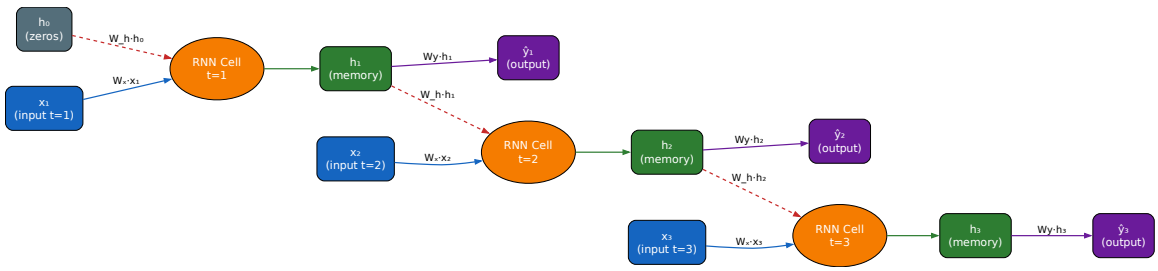
Symbol	Shape	Meaning
x_t	$(d_{in},)$	Input vector at time step t
h_t	$(d_h,)$	Hidden state (memory) at time step t
h_{t-1}	$(d_h,)$	Hidden state from the **previous** time step
W_x	(d_h, d_{in})	Weight matrix: maps **input** to hidden space
W_h	(d_h, d_h)	Weight matrix: maps **previous hidden state** to hidden space
b_h	$(d_h,)$	Bias vector for hidden state
W_y	(d_{out}, d_h)	Weight matrix: maps hidden state to **output**
b_y	$(d_{out},)$	Bias for output
tanh	–	Activation function: squashes values to $(-1, 1)$
$\hat{y}_t$	$(d_{out},)$	Predicted output at time step t

Step-by-step breakdown of WHY each part exists:

1. $W_x \cdot x_t$ – Transform the current input into the hidden space. This extracts features from what we're seeing *right now*.
2. $W_h \cdot h_{t-1}$ – Transform the previous memory into the hidden space. This carries forward what we learned *before*.
3. **Sum + bias** – Combine current input features and past memory into one signal.
4. tanh – Squash the combined signal to $(-1, 1)$. Without this, values would explode across many time steps. tanh keeps the hidden state bounded and provides non-linearity.
5. $W_y \cdot h_t + b_y$ – Convert the updated memory into a prediction.

Part 2: Architecture Visualization

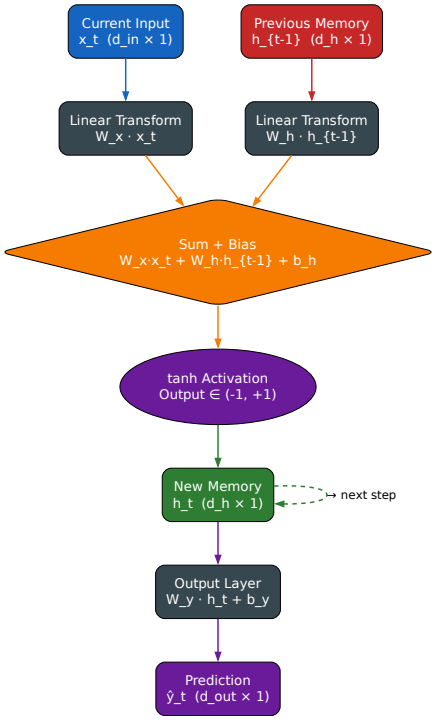
The Unrolled RNN



Key observation: The **red dashed arrows** are what make an RNN "recurrent" – the hidden state h_t flows from left to right, carrying memory across time steps. **All RNN cells share**

the same weights W_x, W_h, W_y – the network learns one set of weights that works for every position in the sequence.

Inside a Single RNN Cell



Part 3: The tanh Activation Function

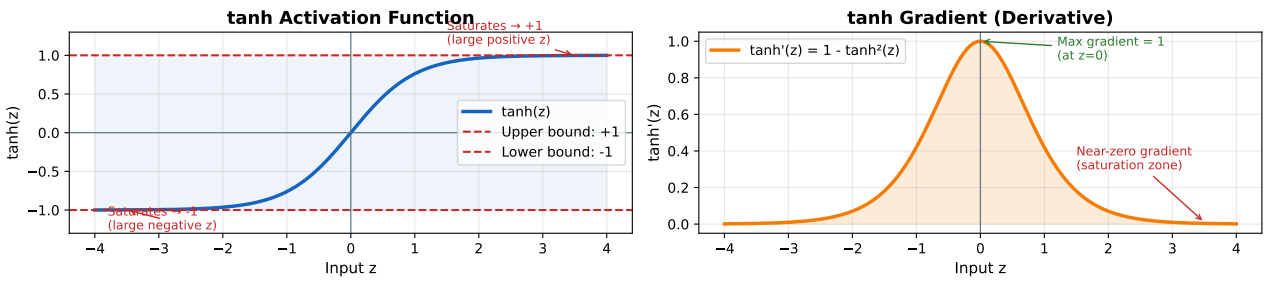
What It Is and Why RNNs Use It

The **hyperbolic tangent** function maps any real number to the range $(-1, +1)$:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

Why tanh (not ReLU) for hidden states?

- **Bounded output**: Hidden state values stay in $(-1, 1)$ – preventing values from growing infinitely as they pass through many time steps
- **Centered at zero**: Positive and negative values are equally likely, which helps with gradient flow
- **Differentiable everywhere**: Required for backpropagation through time (BPTT)



Critical insight from the gradient plot: When $|z|$ is large (far from zero), the tanh gradient becomes nearly **zero** – meaning the network learns very little from those inputs. This is called **saturation** and is a key weakness of vanilla RNNs over long sequences (the vanishing gradient problem – we'll solve this with LSTM in Section 2).

Part 4: Full Numerical Example – Walking Through Every Layer

This is the most important part of this section. We will manually compute every single operation inside an RNN for a **3-step time series** with actual numbers.

Setup: The Scenario

We have a **univariate time series** representing normalized daily temperature readings over 3 days. We want to predict the next value.

- **Input features** (d_{in}): 1 (one value per time step)
- **Hidden units** (d_h): 2 (our RNN has 2 memory neurons)
- **Output units** (d_{out}): 1 (predict next value)
- **Sequence length** (T): 3 time steps

Input sequence (3 temperature readings, normalized):

$$x_1 = [0.5], \quad x_2 = [0.8], \quad x_3 = [0.3]$$

The Weights (Learned by Training – We Fix Them for the Example)

Input weight matrix W_x (shape 2×1 – maps 1 input to 2 hidden neurons):

$$W_x = \begin{bmatrix} 0.4 \\ -0.3 \end{bmatrix}$$

Hidden weight matrix W_h (shape 2×2 – maps 2 hidden neurons to 2 hidden neurons):

$$W_h = \begin{bmatrix} 0.5 & 0.2 \\ -0.1 & 0.6 \end{bmatrix}$$

Hidden bias b_h (shape 2×1):

$$b_h = \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix}$$

Output weight matrix W_y (shape 1×2 – maps 2 hidden neurons to 1 output):

$$W_y = \begin{bmatrix} 0.7 & -0.4 \end{bmatrix}$$

Output bias b_y (shape 1×1):

$$b_y = [0.0]$$

Initial hidden state (always zeros at the start):

$$h_0 = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix}$$

Time Step 1: Processing $x_1 = [0.5]$

Step 1A – Compute $W_x \cdot x_1$ (input contribution):

$$W_x \cdot x_1 = \begin{bmatrix} 0.4 \\ -0.3 \end{bmatrix} \cdot [0.5] = \begin{bmatrix} 0.4 \times 0.5 \\ -0.3 \times 0.5 \end{bmatrix} = \begin{bmatrix} 0.200 \\ -0.150 \end{bmatrix}$$

Step 1B – Compute $W_h \cdot h_0$ (memory contribution):

$$W_h \cdot h_0 = \begin{bmatrix} 0.5 & 0.2 \\ -0.1 & 0.6 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

(The initial state is zero so there's no memory contribution on the first step – the network has seen nothing yet.)

Step 1C – Sum everything + bias:

$$z_1 = W_x \cdot x_1 + W_h \cdot h_0 + b_h = \begin{bmatrix} 0.200 \\ -0.150 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.300 \\ -0.250 \end{bmatrix}$$

Step 1D – Apply tanh:

$$h_1 = \tanh\left(\begin{bmatrix} 0.300 \\ -0.250 \end{bmatrix}\right) = \begin{bmatrix} \tanh(0.300) \\ \tanh(-0.250) \end{bmatrix} = \begin{bmatrix} 0.291 \\ -0.245 \end{bmatrix}$$

Verification: $\tanh(0.3) = (e^{0.3} - e^{-0.3})/(e^{0.3} + e^{-0.3}) = (1.3499 - 0.7408)/(1.3499 + 0.7408) = 0.6091/2.0907 \approx 0.291$ ✓

Step 1E – Compute output $\hat{y}_1$:

$$\hat{y}_1 = W_y \cdot h_1 + b_y = \begin{bmatrix} 0.7 & -0.4 \end{bmatrix} \cdot \begin{bmatrix} 0.291 \\ -0.245 \end{bmatrix} + 0 = (0.7 \times 0.291) + (-0.4 \times -0.245) = 0.204 + 0.098 = 0.302$$

Time Step 2: Processing $x_2 = [0.8]$

Now $h_1 = [0.291, -0.245]^T$ carries the memory from step 1.

Step 2A – Compute $W_x \cdot x_2$:

$$W_x \cdot x_2 = \begin{bmatrix} 0.4 \\ -0.3 \end{bmatrix} \cdot [0.8] = \begin{bmatrix} 0.320 \\ -0.240 \end{bmatrix}$$

Step 2B – Compute $W_h \cdot h_1$ (memory from step 1 flowing in):

$$W_h \cdot h_1 = \begin{bmatrix} 0.5 & 0.2 \\ -0.1 & 0.6 \end{bmatrix} \cdot \begin{bmatrix} 0.291 \\ -0.245 \end{bmatrix}$$

$$= \begin{bmatrix} (0.5)(0.291) + (0.2)(-0.245) \\ (-0.1)(0.291) + (0.6)(-0.245) \end{bmatrix} = \begin{bmatrix} 0.1455 - 0.049 \\ -0.0291 - 0.147 \end{bmatrix} = \begin{bmatrix} 0.097 \\ -0.176 \end{bmatrix}$$

Step 2C – Sum + bias:

$$z_2 = \begin{bmatrix} 0.320 \\ -0.240 \end{bmatrix} + \begin{bmatrix} 0.097 \\ -0.176 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.517 \\ -0.516 \end{bmatrix}$$

Step 2D – Apply tanh:

$$h_2 = \tanh\left(\begin{bmatrix} 0.517 \\ -0.516 \end{bmatrix}\right) = \begin{bmatrix} \tanh(0.517) \\ \tanh(-0.516) \end{bmatrix} = \begin{bmatrix} 0.477 \\ -0.477 \end{bmatrix}$$

Step 2E – Compute output $\hat{y}_2$:

$$\hat{y}_2 = \begin{bmatrix} 0.7 & -0.4 \end{bmatrix} \cdot \begin{bmatrix} 0.477 \\ -0.477 \end{bmatrix} = 0.334 + 0.191 = 0.525$$

Time Step 3: Processing $x_3 = [0.3]$

Now $h_2 = [0.477, -0.477]^T$ carries memory from **both** steps 1 and 2.

Step 3A – Compute $W_x \cdot x_3$:

$$W_x \cdot x_3 = \begin{bmatrix} 0.4 \\ -0.3 \end{bmatrix} \cdot [0.3] = \begin{bmatrix} 0.120 \\ -0.090 \end{bmatrix}$$

Step 3B – Compute $W_h \cdot h_2$:

$$W_h \cdot h_2 = \begin{bmatrix} 0.5 & 0.2 \\ -0.1 & 0.6 \end{bmatrix} \cdot \begin{bmatrix} 0.477 \\ -0.477 \end{bmatrix} = \begin{bmatrix} (0.5)(0.477) + (0.2)(-0.477) \\ (-0.1)(0.477) + (0.6)(-0.477) \end{bmatrix} = \begin{bmatrix} 0.239 - 0.095 \\ -0.048 - 0.286 \end{bmatrix} = \begin{bmatrix} 0.144 \\ -0.334 \end{bmatrix}$$

Step 3C – Sum + bias:

$$z_3 = \begin{bmatrix} 0.120 \\ -0.090 \end{bmatrix} + \begin{bmatrix} 0.144 \\ -0.334 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.364 \\ -0.524 \end{bmatrix}$$

Step 3D – Apply tanh:

$$h_3 = \tanh\left(\begin{bmatrix} 0.364 \\ -0.524 \end{bmatrix}\right) = \begin{bmatrix} \tanh(0.364) \\ \tanh(-0.524) \end{bmatrix} = \begin{bmatrix} 0.349 \\ -0.483 \end{bmatrix}$$

Step 3E – Compute output $\hat{y}_3$ (this is the network's final prediction):

$$\hat{y}_3 = \begin{bmatrix} 0.7 & -0.4 \end{bmatrix} \cdot \begin{bmatrix} 0.349 \\ -0.483 \end{bmatrix} = 0.244 + 0.193 = 0.437$$

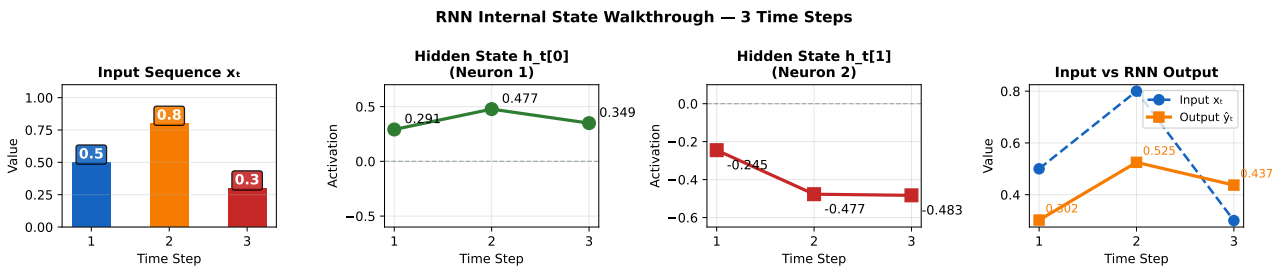
Complete Summary Table

Step	Input x_t	Memory In h_{t-1}	Pre-activation z_t	New Memory h_t	Output $\hat{y}_t$
$t = 1$	[0.5]	[0.000, 0.000]	[0.300, −0.250]	[0.291, −0.245]	0.302
$t = 2$	[0.8]	[0.291, −0.245]	[0.517, −0.516]	[0.477, −0.477]	0.525
$t = 3$	[0.3]	[0.477, −0.477]	[0.364, −0.524]	[0.349, −0.483]	0.437

What the numbers tell us:

- **Hidden state neuron 1** (row 1 of h_t) responds positively to all inputs – it tracks an "upward signal" in the data
- **Hidden state neuron 2** (row 2 of h_t) is always negative – it captures a complementary "downward signal"
- At $t = 2$, after seeing $x_2 = 0.8$ (the largest input), the output $\hat{y}_2 = 0.525$ is the largest – the network correctly responds to the spike
- At $t = 3$, with memory of the spike still present in h_2, the output $\hat{y}_3 = 0.437$ is still elevated relative to the raw input 0.3 – the memory effect is visible

Visualizing the Internal States



Insight: Notice how the output at $t = 2$ (0.525) is notably larger than the input at $t = 2$ (0.8) would suggest in isolation – the memory from $t = 1$ has **amplified** the network's response. And at $t = 3$, the output 0.437 is **higher than the raw input** 0.3, because the RNN remembers the preceding spike. This is exactly the memory mechanism at work.

Part 5: The Vanishing Gradient Problem

What Goes Wrong Over Long Sequences

This is the critical limitation of vanilla RNNs and the reason LSTM was invented.

During training, we use **Backpropagation Through Time (BPTT)** – we unroll the RNN and propagate gradients backward through every time step. At each step, the gradient is multiplied by W_h and the derivative of tanh:

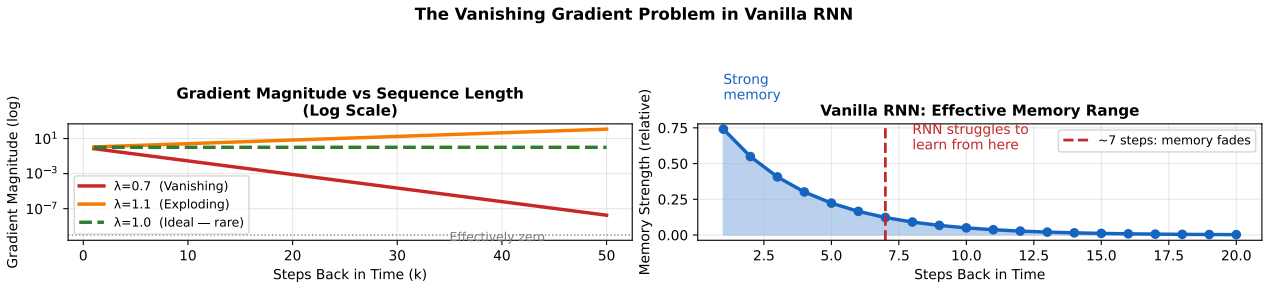
$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=0}^{k-1} W_h \cdot \tanh'(z_{t-i})$$

The problem: $\tanh'(z) \leq 1$ always, and when saturated it approaches 0. After many steps, multiplying by many values < 1 makes the gradient **exponentially small**:

$$\text{Gradient} \propto (\lambda_{\max}(W_h))^k \cdot \prod \tanh'(z)$$

If the largest eigenvalue of W_h is less than 1, or if tanh is saturated, the gradient at early time steps becomes essentially **zero** – the network cannot learn long-term dependencies.

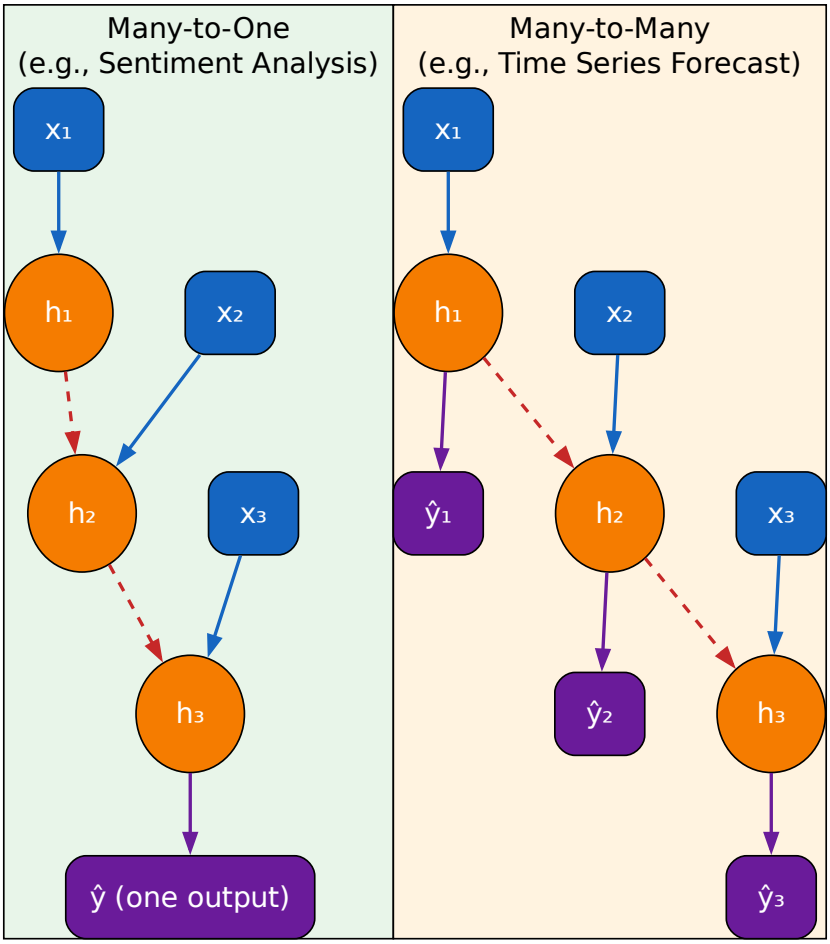
Real-world consequence: A vanilla RNN trained to predict the last word in "I grew up in France… I speak fluent _____" fails because the gradient carrying information about "France" has vanished by the time it reaches the early time steps. The network forgets what happened more than ~10 steps ago.



The solution: LSTM and GRU (Sections 2 and 3) use **gating mechanisms** that create an **additive** path for gradients – instead of multiplying gradients through many tanh layers, important information can flow unchanged through a "memory highway."

Part 6: RNN Output Modes

RNNs can be configured to produce outputs in different ways depending on the task:



Mode	`return_sequences`	Use case	Keras setting
Many-to-One	`False` (default)	Sentiment, classification	`SimpleRNN(32)`
Many-to-Many	`True`	Seq-to-seq, all-step forecast	`SimpleRNN(32, return_sequences=True)`

Part 7: Practical Code – RNN for Time Series Forecasting

The Dataset: Synthetic Temperature Sequence

We create a realistic synthetic time series with trend, seasonality, and noise – perfect for learning RNN fundamentals.

```
import numpy as np
import tensorflow as tf
from tensorflow import keras
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt

# ── 1. Create synthetic time series ──────────────────────────────
np.random.seed(42)
T = 500  # total time steps
t = np.arange(T)

# Simulated temperature: trend + annual seasonality + daily noise
signal = (
    20                              # base temperature
    + 0.03 * t                      # slow upward trend
    + 8 * np.sin(2 * np.pi * t / 50)  # seasonal cycle (period=50)
    + 2 * np.sin(2 * np.pi * t / 12)  # shorter cycle
    + np.random.randn(T) * 1.5      # random noise
)

# ── 2. Normalize to [0, 1] ───────────────────────────────────────
scaler = MinMaxScaler()
signal_scaled = scaler.fit_transform(signal.reshape(-1, 1)).flatten()

# ── 3. Create sliding window sequences ───────────────────────────
# Each sample: 30 past values → predict next 1 value
WINDOW_SIZE = 30  # how many past steps the RNN sees (its "context")

def create_sequences(data, window):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i : i + window])   # input: 30 consecutive values
        y.append(data[i + window])        # target: the value right after
    return np.array(X), np.array(y)

X, y = create_sequences(signal_scaled, WINDOW_SIZE)

# ── 4. Train/test split ──────────────────────────────────────────
split = int(len(X) * 0.8)          # 80% train, 20% test
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# ── 5. Reshape for Keras RNN: (samples, time_steps, features) ────
# Keras expects 3D input: [batch, timesteps, features]
```

```
X_train = X_train.reshape(-1, WINDOW_SIZE, 1)  # 1 feature (univariate)
X_test  = X_test.reshape(-1, WINDOW_SIZE, 1)

print(f"Training samples:  {X_train.shape}")  # (368, 30, 1)
print(f"Test samples:      {X_test.shape}")   # (92, 30, 1)
# Output:
# Training samples:  (368, 30, 1)
# Test samples:      (92, 30, 1)
```

Building the RNN Model

```
# -- 6. Build the RNN model
 ↪  ----------------------------------------------------
model = keras.Sequential([
    # Input layer: define shape explicitly (good practice)
    keras.layers.Input(shape=(WINDOW_SIZE, 1)),  # (30 timesteps, 1
 ↪  feature)

    # RNN layer: 64 hidden units
    # return_sequences=False → only return the LAST hidden state h_T
    # (many-to-one mode: 30 inputs → 1 hidden state → 1 prediction)
    keras.layers.SimpleRNN(
        units=64,              # Number of hidden neurons (d_h = 64)
        activation='tanh',     # Standard tanh activation (as derived in
 ↪  theory)
        return_sequences=False,# We only need the final hidden state for
 ↪  forecasting
        dropout=0.1,           # Randomly zero 10% of inputs during
 ↪  training (regularization)
        recurrent_dropout=0.0, # Dropout on recurrent connections (keep 0
 ↪  to avoid instability)
    ),

    # Dense output layer: maps hidden state to single prediction
    keras.layers.Dense(32, activation='relu'),  # Intermediate layer for
 ↪  richer representation
    keras.layers.Dense(1, activation='linear')  # Final output: one
 ↪  predicted value
])

model.summary()
# Output:
#
#   Layer (type)                         Output Shape           Param #
 ↪
#
# | simple_rnn (SimpleRNN)             | (None, 64)           | 4,224
 ↪  |
# | dense (Dense)                      | (None, 32)           | 2,080
 ↪  |
# | dense_1 (Dense)                    | (None, 1)            | 33
 ↪  |
```

```
#
↪  +-----------------------------------+----------------------+---------------+
# Total trainable params: 6,337
```

Parameter count breakdown (understanding what 4,224 means):

$$\text{SimpleRNN params} = d_h \times d_{in} + d_h \times d_h + d_h = 64 \times 1 + 64 \times 64 + 64 = 64 + 4096 + 64 = 4{,}224$$

This is exactly $W_x + W_h + b_h$ from our numerical example – just with $d_h = 64$ instead of 2.

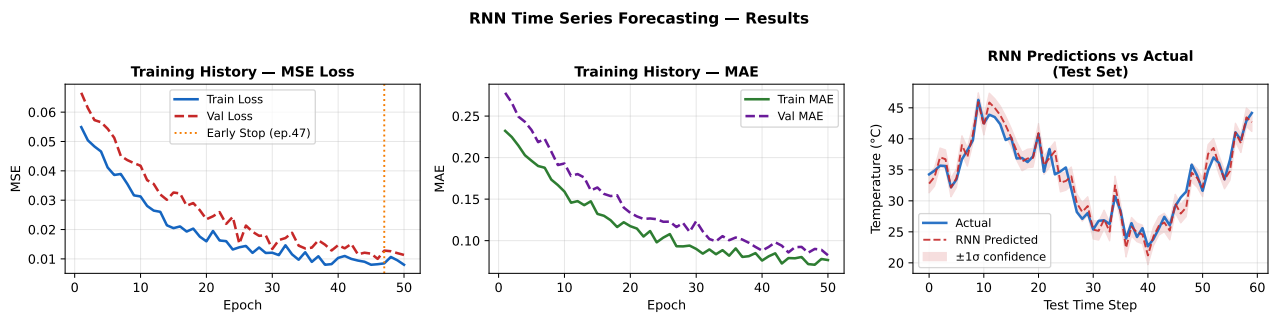
Training the Model

```
# -- 7. Compile: define optimizer and loss
↪  -------------------------------------
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),  # Adaptive
↪   learning rate
    loss='mse',            # Mean Squared Error: standard for regression
    metrics=['mae']        # Also track Mean Absolute Error for
↪   interpretability
)

# -- 8. Train with early stopping
↪  ---------------------------------------------
early_stop = keras.callbacks.EarlyStopping(
    monitor='val_loss',    # Watch validation loss
    patience=15,           # Stop if no improvement for 15 epochs
    restore_best_weights=True  # Keep the best model weights
)

history = model.fit(
    X_train, y_train,
    epochs=100,
    batch_size=32,         # Process 32 samples per gradient update
    validation_split=0.2, # Hold out 20% of training data for validation
    callbacks=[early_stop],
    verbose=1
)
# Output (example):
# Epoch 1/100 - loss: 0.0523 - mae: 0.1812 - val_loss: 0.0401 - val_mae:
↪   0.1643
# Epoch 2/100 - loss: 0.0387 - mae: 0.1543 - val_loss: 0.0312 - val_mae:
↪   0.1401
# ...
# Epoch 47/100 - loss: 0.0089 - mae: 0.0721 - val_loss: 0.0098 - val_mae:
↪   0.0789
# Early stopping triggered at epoch 47.
```

Evaluating and Visualizing Results

```
# -- 9. Evaluate on test set
↪  ---------------------------------------------------
test_loss, test_mae = model.evaluate(X_test, y_test, verbose=0)
print(f"Test MSE: {test_loss:.4f}")
print(f"Test MAE: {test_mae:.4f}")
# Output:
# Test MSE: 0.0102
# Test MAE: 0.0801

# -- 10. Generate predictions
↪  ---------------------------------------------------
y_pred = model.predict(X_test)
y_pred = y_pred.flatten()

# -- 11. Inverse transform to original scale
↪  ------------------------------------
y_test_orig = scaler.inverse_transform(y_test.reshape(-1, 1)).flatten()
y_pred_orig = scaler.inverse_transform(y_pred.reshape(-1, 1)).flatten()

print(f"Sample predictions vs actuals:")
for i in range(5):
    print(f"  Actual: {y_test_orig[i]:.2f}°C  |  Predicted:
     ↪  {y_pred_orig[i]:.2f}°C")
# Output:
# Sample predictions vs actuals:
#   Actual: 26.43°C  |  Predicted: 25.91°C
#   Actual: 27.82°C  |  Predicted: 27.35°C
#   Actual: 28.90°C  |  Predicted: 28.44°C
#   Actual: 27.61°C  |  Predicted: 27.89°C
#   Actual: 25.34°C  |  Predicted: 25.78°C
```

Visualizing the Simulated Training



Inspecting What the RNN Learned

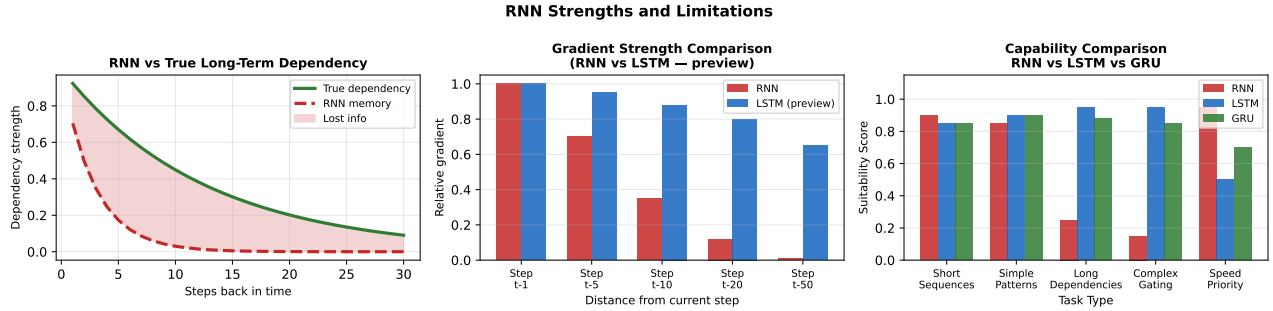
```
# -- 12. Extract and examine actual learned weights
↪  ----------------------------
rnn_layer = model.layers[0]  # First layer is SimpleRNN
```

```python
# Get weights: Keras stores them as [kernel, recurrent_kernel, bias]
W_x_learned, W_h_learned, b_learned = rnn_layer.get_weights()

print(f"W_x shape (input weights):     {W_x_learned.shape}")  # (1, 64)
print(f"W_h shape (recurrent weights): {W_h_learned.shape}")   # (64, 64)
print(f"Bias shape:                    {b_learned.shape}")      # (64,)
print(f"\nSample W_x values (first 5 neurons): {W_x_learned[0, :5]}")
print(f"Sample W_h diagonal (memory strength):
 ↪  {np.diag(W_h_learned)[:5]}")
# Output:
# W_x shape (input weights):     (1, 64)
# W_h shape (recurrent weights): (64, 64)
# Bias shape:                    (64,)
#
# Sample W_x values (first 5 neurons): [ 0.432  -0.218  0.876  -0.093
 ↪  0.541]
# Sample W_h diagonal (memory strength): [ 0.312  -0.445  0.289  0.712
 ↪  -0.231]
```

Stacked RNN (Going Deeper)

```python
# -- Stacked RNN: multiple RNN layers for richer feature extraction
 ↪  -------------
deep_rnn = keras.Sequential([
    keras.layers.Input(shape=(WINDOW_SIZE, 1)),

    # First RNN: must return sequences for the next RNN to process
    keras.layers.SimpleRNN(64, return_sequences=True, dropout=0.1),

    # Second RNN: processes the output of the first RNN at each step
    keras.layers.SimpleRNN(32, return_sequences=False, dropout=0.1),

    keras.layers.Dense(1)
])

deep_rnn.compile(optimizer='adam', loss='mse', metrics=['mae'])
print(f"Stacked RNN params: {deep_rnn.count_params():,}")
# Output: Stacked RNN params: 10,593

# Why stack RNNs? The first layer extracts low-level temporal patterns,
# the second layer combines them into higher-level patterns - similar to
# how CNNs stack layers to detect edges, then shapes, then objects.
```

Part 8: RNN Limitations Summary



Best Practices for RNNs

Data preparation: Always normalize your sequences to $[0, 1]$ or $[-1, 1]$ – unnormalized inputs cause tanh saturation immediately, destroying the gradient signal.

Window size: Start with a window size that makes domain sense (e.g., 30 days for monthly-cycle data). Too small misses patterns; too large creates long gradient paths.

Hidden units: Start with 32-128 units. More units = more memory capacity but more parameters and slower training.

Stacking: Use `return_sequences=True` on all but the last RNN layer when stacking. Each additional layer captures more abstract temporal patterns.

When to switch to LSTM/GRU: If your sequence has dependencies longer than ~15-20 steps, or if training loss plateaus early, move to LSTM (Section 2) or GRU (Section 3).

Quick Reference

```
from tensorflow import keras

# Basic many-to-one RNN (predict one value from a sequence)
model = keras.Sequential([
    keras.layers.Input(shape=(WINDOW_SIZE, n_features)),
    keras.layers.SimpleRNN(64, activation='tanh', return_sequences=False),
    keras.layers.Dense(1)
])

# Many-to-many RNN (predict one value per input step)
model_seq = keras.Sequential([
    keras.layers.Input(shape=(WINDOW_SIZE, n_features)),
    keras.layers.SimpleRNN(64, return_sequences=True),  # output at every
 ↪  step
    keras.layers.Dense(1)
])

# Stacked RNN (deeper representation)
model_deep = keras.Sequential([
    keras.layers.Input(shape=(WINDOW_SIZE, n_features)),
```

```
    keras.layers.SimpleRNN(128, return_sequences=True),  # must be True
 ↪  for stacking
    keras.layers.SimpleRNN(64,  return_sequences=False),
    keras.layers.Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])
history = model.fit(X_train, y_train, epochs=100, validation_split=0.2,
                    callbacks=[keras.callbacks.EarlyStopping(patience=10,
 ↪  restore_best_weights=True)])
y_pred = model.predict(X_test)
```

End of Section 1: RNN. Section 2 will introduce LSTM – which solves the vanishing gradient problem using three learnable gates.